GUIDO

Guids your way to a perfect project

Explainable Clickbait Detection System

AI-Powered Detection with LIME Interpretability

Project Documentation & Setup Guide

Generated on: March 23, 2026

1. Introduction

Clickbait headlines are designed to attract attention and encourage visitors to click on a link to a particular web page. While effective for marketing, they often lead to low-quality content, misinformation, and a frustrating user experience. Traditional detection methods rely on manual flagging or simple keyword matching, which are easily bypassed.

The Explainable Clickbait Detection System is a complete full-stack Artificial Intelligence solution that uses Natural Language Processing (NLP) to detect deceptive headlines automatically in real-time.

The primary features of this system include:

- Accurate Local AI Detection utilizing a fine-tuned DistilBERT transformer running entirely on your local machine.
- Explainable AI (XAI) integration using LIME (Local Interpretable Model-Agnostic Explanations) to show exactly which words caused the AI to flag the title.
- Privacy-Focused architecture because all inference runs locally via FastAPI without uploading browsing habits to the cloud.
- A Chrome Extension with a premium Liquid Glass aesthetic designed to scan, highlight, and explain clickbait dynamically on any website.
- A Web Application hosting a manual testing portal featuring real-time Chart.js visual analytics and domain tracking.
- A Wall of Shame Dashboard and Digital Wellbeing blur modes to protect users from deceptive psychological tactics.

2. Objectives of the Project

The main objectives of this project are:

- To develop a highly accurate AI model capable of distinguishing clickbait from legitimate news.
- To provide transparency by integrating explainability so users understand why the model made a specific prediction.

- To process everything locally, guaranteeing user privacy while scanning their browsing history.
- To create a seamless browser extension that works directly on standard news portals and search engines.
- To track analytics and present them via simple, intuitive interfaces mapping deceptive behaviors.

3. Real-World Applications

This framework offers substantial utility beyond generic academic testing. Real-world applications include:

- News filtering systems eliminating low-effort content from user feeds.
- Social media monitoring for combating click-fraud and sensationalism.
- Digital wellbeing tools mitigating psychological exhaustion caused by manipulative phraseology.
- Journalism credibility analysis, establishing automated trust scores based on semantic integrity.

4. Technology Stack

4.1 Python, FastAPI & PyTorch

Python is the primary language running the backend intelligence. PyTorch is the machine learning framework running the tensors, and FastAPI acts as the ultra-fast asynchronous REST API interface fielding extension requests.

4.2 DistilBERT Transformer

A highly refined, smaller version of BERT (Bidirectional Encoder Representations from Transformers). DistilBERT offers up to 60% faster inference while retaining 97% of BERT's natural language understanding. The model is specifically fine-tuned on clickbait datasets.

4.3 LIME (Explainable AI)

Local Interpretable Model-Agnostic Explanations (LIME) mathematically perturbs the input sentence and evaluates how the DistilBERT probabilities shift. It constructs a visual heatmap highlighting which specific words pushed the prediction towards Clickbait or Safe.

4.4 Browser Extension API (Manifest V3)

Using raw Javascript, CSS, and HTML under Chrome's Manifest V3 architecture. Background service workers proxy the local API requests to bypass strict site Content Security Policies (CSP), while content scripts dynamically manipulate DOM header elements.

4.5 Web UI & Chart.js

The standalone testing portal is built with raw stylized CSS glassmorphism, completely devoid of complex frontend frameworks for maximum performance. Chart.js is utilized for compiling raw JSON statistics into beautiful interactive graphs.

5. System Architecture

The architecture pipelines the NLP task across multiple detached components:

- Data Initialization Module (Fine-Tuning Script)
- Inference Endpoint (FastAPI Server)
- Explainability Engine (LIME Interpreter)
- Web Client Portal (Input Web App)
- Browser Injection (Chrome Extension)

The system works by extracting headlines dynamically from webpages and sending them to a local FastAPI server, where the fine-tuned DistilBERT model executes classification. The LIME module instantly generates localized explanations based on the tensor probabilities, and the structured verdict is returned asynchronously to the browser extension for real-time DOM visualization.

This modular pipeline ensures that the machine-learning execution is completely physically isolated from the web-rendering thread, preventing freezing and ensuring optimal multi-core utilization.

6. Installation & Prerequisites

To initialize the framework, it is required to use a Python version 3.8+ capable environment.

- 1. Clone/Download the project framework folder.
- 2. Create a virtual environment: python -m venv .venv
- 3. Activate it: .\.venv\Scripts ctivate
- 4. Install modules: pip install -r requirements.txt

7. Model Training & Backend Execution

7.1 Training the Local DistilBERT Model

Prior to execution, the local weights must be trained. Ensure 'clickbait_data.csv' is saved containing standard 'headline' and 'label' columns inside the model directory.

Execute: python model/train.py

The script handles dataset splitting, tokenization, multi-epoch processing via hardware acceleration, performance metric evaluations, and final exporting to the /local_model folder.

7.2 Starting the FastAPI Server

Once weights generate successfully, launch the REST inference endpoint.

Execute: uvicorn backend.app:app --host 0.0.0.0 --port 8000

The system automatically parses the raw safetensors into memory alongside the LIME explainer setup.

8. Advanced Extension Integrations

The Chrome extension dynamically parses heading elements (H1, H2, class='title', etc.) currently loaded in the viewport array. It batches queries to the localhost:8000 pipeline in chunks of 20 to prevent pipeline stalling and renders color-coded verdict outlines (Red/Green).

Wall of Shame Dashboard

The platform utilizes chrome.storage.local to map statistical domains historically. Tracking which corporate entities output the highest ratio of deceptive titles globally against the baseline testing average, visualized in localized interactive graphs.

Digital Wellbeing Mode

A CSS blur filter that actively censors identified clickbait terminology until physically hovered. Designed specifically to remove the psychological stress of deceptive FOMO (Fear Of Missing Out) tactics deployed structurally.

9. VIVA QUESTIONS AND ANSWERS

PART 1: AI MODEL AND ARCHITECTURE

Q1: Why did you choose DistilBERT over standard BERT or basic Logistic Regression?

Logistic regression struggles with deep contextual associations in modern deceptive sentences. Standard BERT is highly accurate but computationally massive. DistilBERT presents the perfect middle-ground: shrinking the parameter size by 40% and accelerating inference by 60%, allowing it to execute instantly on local CPUs continuously for browser plugins.

Q2: How does LIME (Explainable AI) function mathematically in your application?

LIME operates by isolating the headline prediction, generating hundreds of perturbed random versions of the text (removing words randomly), and submitting them backward into the DistilBERT inference engine. By correlating which removed words drastically dropped the Clickbait probability, LIME maps out the exact linguistic importance of individual terms.

Q3: Why run the model entirely locally? Why not use an external Cloud API?

If a browser extension parses and uploads every single headline a user renders across their entire internet browsing history to a central cloud server, it represents a massive invasion of privacy. Running processing locally on port 8000 ensures maximum security, zero ongoing server expenditure, and sub-millisecond network latency.

Q4: How does the Chrome extension bypass standard Content Security Policies (CSP)?

Modern browsers block injected Content Scripts from making raw fetch requests to arbitrary IP ports (like localhost). To bypass this, the Content Script forwards a messaging payload to the internal Background Service Worker (background.js). The Service Worker acts as an invisible proxy with elevated permissions, executing the API fetch and delivering the transparent output backward to the DOM rendering loop.

PART 2: DEVELOPMENT AND ANALYTICS

Q5: How did you mitigate visual clutter if the extension flags multiple headlines?

Instead of immediately rendering massive LIME explanation boxes everywhere, the system utilizes 'Hover-to-Explain' delays. It simply outlines the specific item in Green or Red. Full visual context and heatmaps only render following a confirmed 800-millisecond hover interaction.

Q6: What is the purpose of the Wall of Shame feature?

It serves as a longitudinal data collection mechanic storing specific website domains mapped against the volume of clickbait generated. This provides users numerical leverage dynamically mapping untrustworthy organizations systematically utilizing deception.

Q7: What is Digital Wellbeing blur?

It actively applies a CSS variable blur to any detected clickbait headline matching the target criteria. The psychological trigger of clickbait works upon peripheral visual engagement. By blurring it out entirely, users are unburdened from the cognitive load of ignoring manipulative phraseology.

Q8: What are the primary system limitations?

The framework cannot detect image-based clickbait (misleading thumbnails without text). It also holds a limited understanding of extreme sarcasm or modern internet slang not inherently parsed inside the localized datasets. Consequently, performance depends heavily on dataset quality, and edge cases may produce false positives for highly emotional but legitimate headlines.

Q9: How can this system be structurally improved in future scoped versions?

Future development cycles could improve overall dataset sizes to guarantee better generalized accuracy. The model could also be customized specifically for domain-specific news sub-sectors (like finance or gaming). Finally, we could optimize raw inference parsing for extremely low-end systems and refine UI responsiveness and dashboard visualization clarity.

Q10: What is Clickbait in NLP terms?

Clickbait is a form of misleading textual content that uses exaggerated, emotional, or curiosity-driven language to attract user attention. In NLP, it is treated as a binary text classification problem, where input text is classified as clickbait or non-clickbait based on learned linguistic patterns.

Q11: What is Tokenization and why is it important?

Tokenization is the process of converting raw text into smaller units (tokens), such as words or subwords. In DistilBERT, subword tokenization is used to handle unknown or rare words efficiently. It is essential because machine learning models require numerical representation and cannot process raw text directly.

Q12: What is the role of the attention mechanism in DistilBERT?

The attention mechanism helps the model focus on important words in a sentence by assigning specific weights to each token. This allows the model to understand context dynamically and relationships between words, vastly improving global classification accuracy.

Q13: What is the exact difference between BERT and DistilBERT?

DistilBERT is a mathematically compressed version of BERT via knowledge distillation. It contains fewer parameters, making it significantly faster and lighter while retaining roughly 97% of the original performance. It is extremely suitable for real-time edge applications like browser extensions.

Q14: What is overfitting and how did you avoid it?

Overfitting occurs when an AI model performs exceptionally well on its specific training data but fails to generalize to unseen real-world data. It was avoided by using techniques like explicit dataset validation splits, hyperparameter tuning, and limiting total training epochs.

Q15: Why is FastAPI used instead of Flask?

FastAPI is fundamentally faster at runtime execution and inherently supports asynchronous operations. This makes it far more suitable for handling dozens of simultaneous real-time classification requests fired from a busy DOM browser extension without blocking.

Q16: What exactly is 'inference' in your system?

Inference is the live procedural phase outlining the process of utilizing the mathematically trained model to make immediate predictions on brand-new, unseen input data (headlines) in real-time.

Q17: What is the specific role of Chart.js in your project setup?

Chart.js is leveraged inside the centralized web dashboard to visualize aggregated JSON analytics - such as clickbait frequency, domain tracking over time, and user interaction patterns - in an interactive, highly visual format.

Q18: What is CSP (Content Security Policy) regarding your extension?

CSP is an ingrained browser security mechanism that restricts where loaded scripts can fetch arbitrary data. In this project, CSP restrictions directly from websites are bypassed safely by utilizing an invisible background service worker to explicitly handle API fetch operations.

Q19: What type of machine learning is used in this model?

The system formally relies on Supervised Learning architectures, where the model adjusts parameters based explicitly on pre-labeled target data mapping definitions of clickbait versus entirely non-clickbait features.